

≡ My Notes

Gaussian processes (1/3) - From scratch

This post explores some concepts behind Gaussian processes, such as stochastic processes and the kernel function. We will build up deeper understanding of Gaussian process regression by implementing them from scratch using Python and NumPy.

This post is followed by [a second post demonstrating how to fit a Gaussian process kernel](#) with [TensorFlow probability](#). This is the first post part of a series on Gaussian processes:

1. [Understanding Gaussian processes \(this\)](#)
2. [Fitting a Gaussian process kernel](#)
3. [Gaussian process kernels](#)

In what follows we assume familiarity with basic probability and linear algebra especially in the context of multivariate Gaussian distributions. Have a look at [this post](#) if you need a refresher on the Gaussian distribution.

```
# Imports
```



What are Gaussian processes? The name implies that it's a stochastic process of random variables with a Gaussian distribution. This might not mean much at this moment so let's dig a bit deeper into its meaning.

Stochastic process

[Stochastic processes](#) typically describe systems randomly changing over time. The processes are [stochastic](#) due to the uncertainty in the system. Even if the starting point is known, there are several directions in which the processes can evolve.

An example of a stochastic process that you might have come across is the model of [Brownian motion](#) (also known as [Wiener process](#)). Brownian motion is the random motion of particles suspended in a fluid. It can be seen as a continuous [random walk](#) where a particle moves around in the fluid due to other particles randomly bumping into it. We can simulate this process over time t in 1 dimension d by starting out at position 0 and moving the particle over a certain amount of time Δt with a random distance Δd from the previous position. The random distance is sampled from a [normal distribution](#) with mean 0 and variance Δt . Sampling Δd from this normal distribution is noted as $\Delta d \sim \mathcal{N}(0, \Delta t)$. The position $d(t)$ at time t evolves as $d(t + \Delta t) = d(t) + \Delta d$.

We simulate 5 different paths of Brownian motion in the following figure, each path is illustrated with a different color.

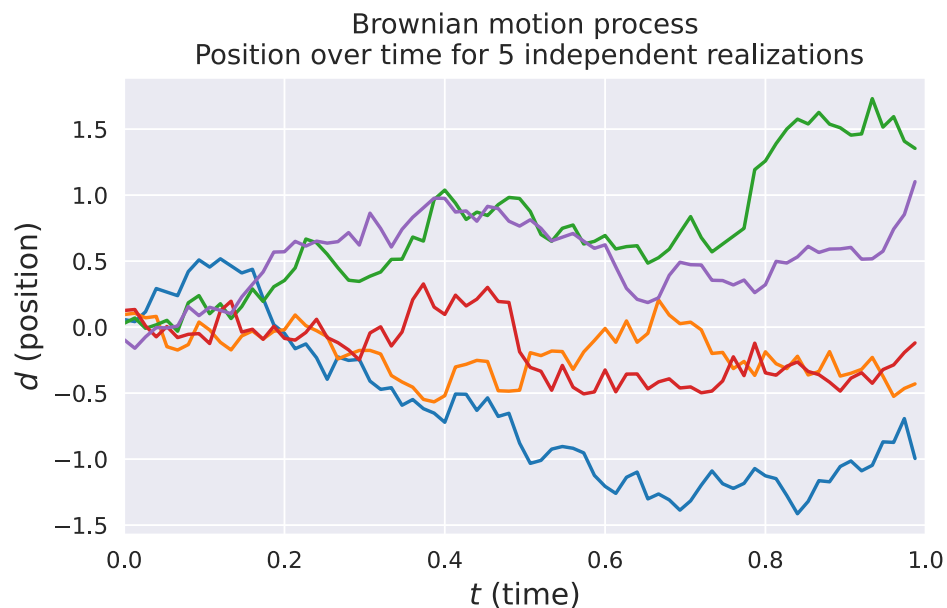
```

# 1D simulation of the Brownian motion process
total_time = 1
nb_steps = 75
delta_t = total_time / nb_steps
nb_processes = 5 # Simulate 5 different motions
mean = 0. # Mean of each movement
stdev = np.sqrt(delta_t) # Standard deviation of each movement

# Simulate the brownian motions in a 1D space by cumulatively
# making a new movement delta_d
distances = np.cumsum(
    # Move randomly from current location to  $N(0, \text{delta\_t})$ 
    np.random.normal(
        mean, stdev, (nb_processes, nb_steps)),
    axis=1)

plt.figure(figsize=(6, 4))
# Make the plots
t = np.arange(0, total_time, delta_t)
for i in range(nb_processes):
    plt.plot(t, distances[i,:])
plt.title((
    'Brownian motion process\n '
    'Position over time for 5 independent realizations'))
plt.xlabel('$t$ (time)', fontsize=13)
plt.ylabel('$d$ (position)', fontsize=13)
plt.xlim([-0, 1])
plt.tight_layout()
plt.show()
#

```



Stochastic processes as distributions over functions

Notice in the figure above that the stochastic process can lead to different paths, also known as [realizations](#) of the process. Each realization defines a position d for every possible timestep t . Every realization thus corresponds to a function $f(t) = d$.

This means that a stochastic process can be interpreted as a random distribution over functions. We can sample a realization of a function from a stochastic process. However each realized function can be different due to the randomness of the stochastic process.

Like the model of Brownian motion, Gaussian processes are stochastic processes. In fact, the Brownian motion process can be reformulated as a Gaussian process [\(3\)](#).

Gaussian processes

Gaussian processes are distributions over functions $f(x)$ of which the distribution is defined by a mean function $m(x)$ and [positive definite](#) covariance function $k(x, x')$, with x the function values and (x, x') all possible pairs in the input [domain](#):

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

where for any finite subset $X = \{\mathbf{x}_1 \dots \mathbf{x}_n\}$ of the domain of x , the [marginal distribution](#) is a [multivariate Gaussian](#) distribution:

$$f(X) \sim \mathcal{N}(m(X), k(X, X))$$

with mean vector $\mu = m(X)$ and covariance matrix $\Sigma = k(X, X)$.

While the multivariate Gaussian captures a finite number of jointly distributed Gaussians, the Gaussian process doesn't have this limitation. Its mean and covariance are defined by a [function](#)). Each input to this function is a variable correlated with the other variables in the input domain, as defined by the covariance function. Since functions can have an infinite input domain, the Gaussian process can be interpreted as an infinite dimensional Gaussian random variable.

Covariance function as prior

To sample functions from the Gaussian process we need to define the mean and covariance functions. The [covariance function](#) $k(x_a, x_b)$ models the joint variability of the Gaussian process random variables. It returns the modelled [covariance](#) between each pair in x_a and x_b .

The specification of this covariance function, also known as the kernel function, implies a distribution over functions $f(x)$. By choosing a specific kernel function k it is possible to set [prior](#) information on this distribution. This kernel function needs to be [positive-definite](#) in order to be a valid covariance function.

In this post we will model the covariance with the [exponentiated quadratic](#) covariance function (also known as the RBF kernel):

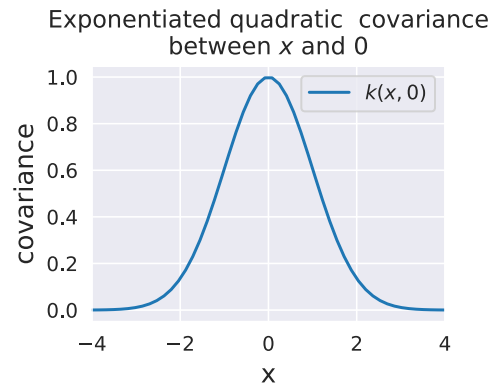
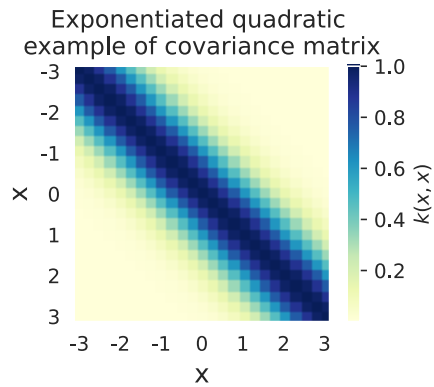
$$k(x_a, x_b) = \exp\left(-\frac{1}{2\sigma^2} \|x_a - x_b\|^2\right)$$

Other kernel functions can be defined resulting in different priors on the Gaussian process distribution. Examples of different kernels are given in a [follow-up post](#).

```
# Define the exponentiated quadratic
def exponentiated_quadratic(xa, xb):
    """Exponentiated quadratic with  $\sigma=1$ """
    # L2 distance (Squared Euclidian)
    sq_norm = -0.5 * scipy.spatial.distance.cdist(xa, xb, 'sqeuclidean')
    return np.exp(sq_norm)
```

An example covariance matrix from the exponentiated quadratic covariance function is plotted in the figure below on the left. The covariance vs input zero is plotted on the right. Note that the exponentiated quadratic covariance decreases exponentially the further away the function values x are from each other.

Illustrate covariance matrix and function



Sampling from prior

In practice we can't just sample a full function evaluation f from a Gaussian process distribution since that would mean evaluating $m(x)$ and $k(x, x')$ at an infinite number of points since x can have an infinite [domain](#). We can however sample function evaluations $\mathbf{y}$ of a function f drawn from a Gaussian process at a finite, but arbitrary, set of points X : $\mathbf{y} = f(X)$.

A finite dimensional subset of the Gaussian process distribution results in a [marginal distribution](#) that is a Gaussian distribution $\mathbf{y} \sim \mathcal{N}(\mu, \Sigma)$ with mean vector $\mu = m(X)$, covariance matrix $\Sigma = k(X, X)$.

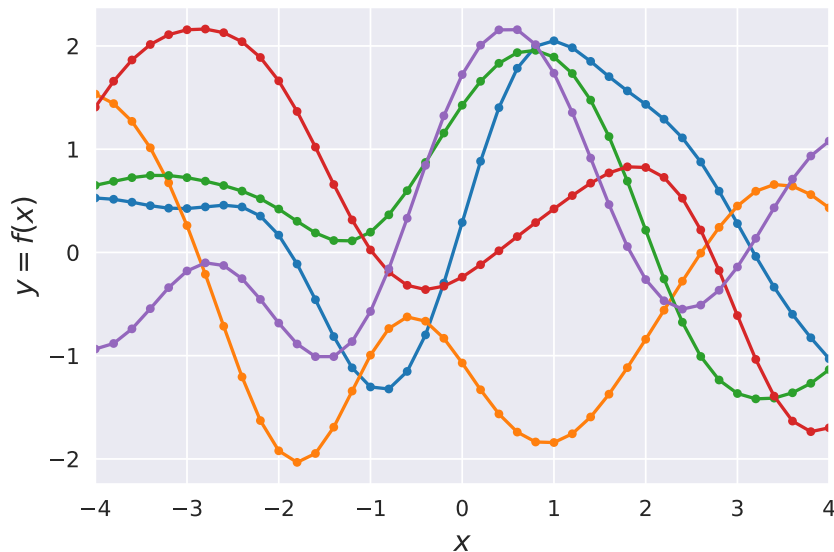
In the figure below we will sample 5 different function realisations from a Gaussian process with exponentiated quadratic prior [\(1\)](#) without any observed data. We do this by drawing correlated samples from a 41-dimensional Gaussian $\mathcal{N}(0, k(X, X))$ with $X = [X_1, \dots, X_{41}]$.

```
# Sample from the Gaussian process distribution
nb_of_samples = 41 # Number of points in each function
number_of_functions = 5 # Number of functions to sample
# Independent variable samples
X = np.expand_dims(np.linspace(-4, 4, nb_of_samples), 1)
Σ = exponentiated_quadratic(X, X) # Kernel of data points

# Draw samples from the prior at our data points.
# Assume a mean of 0 for simplicity
ys = np.random.multivariate_normal(
    mean=np.zeros(nb_of_samples), cov=Σ,
    size=number_of_functions)
```

```
# Plot the sampled functions
plt.figure(figsize=(6, 4))
for i in range(number_of_functions):
    plt.plot(X, ys[i], linestyle='-', marker='o', markersize=3)
plt.xlabel('$x$', fontsize=13)
plt.ylabel('$y = f(x)$', fontsize=13)
plt.title((
    '5 different function realizations at 41 points\n'
    'sampled from a Gaussian process with exponentiated quadra'
plt.xlim([-4, 4])
plt.show()
#
```

5 different function realizations at 41 points
sampled from a Gaussian process with exponentiated quadratic kernel



Another way to visualise this is to take only 2 dimensions of this 41-dimensional Gaussian and plot some of it's 2D marginal distributions.

The next figure on the left visualizes the 2D distribution for $X = [0, 0.2]$ where the covariance $k(0, 0.2) = 0.98$. The figure on the right visualizes the 2D distribution for $X = [0, 2]$ where the covariance $k(0, 2) = 0.14$.

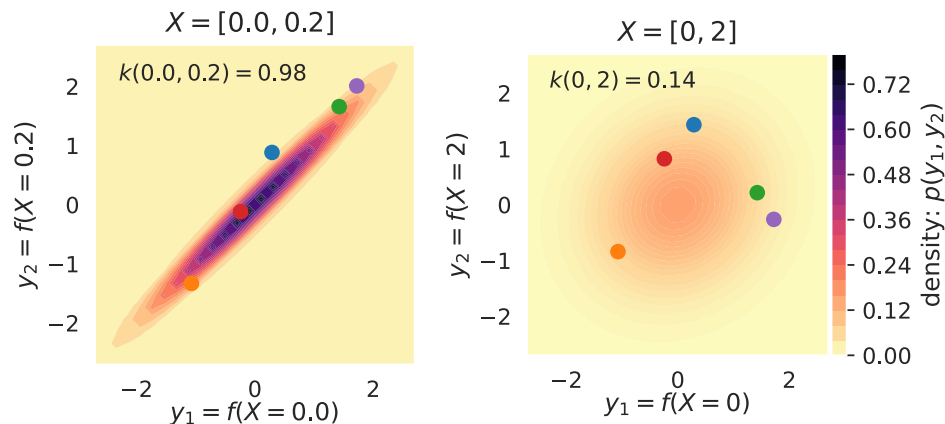
For each of the 2D Gaussian marginals the corresponding samples from the function realisations above have been plotted as colored dots on the figure.

Observe that points close together in the input domain of x are strongly correlated (y_1 is close to y_2), while points further away from each other are almost independent. This is because these marginals come from a Gaussian process with as prior the exponentiated quadratic covariance, which adds prior information that points close to each other in the input space X must be close to each other in the output space y .

Show marginal 2D Gaussians



2D marginal: $y \sim \mathcal{N}(0, k(X, X))$



Gaussian processes for regression

Since Gaussian processes model distributions over functions we can use them to build [regression](#) models. We can treat the Gaussian process as a prior defined by the kernel function and create a [posterior distribution](#) given some data. This posterior distribution can then be used to predict the expected value and probability of the output variable y given input variables X .

Predictions from posterior

We want to make predictions $y_2 = f(X_2)$ for n_2 new samples, and we want to make these predictions based on our Gaussian process prior and n_1 previously observed data points (X_1, y_1) . This can be done with the help of the posterior distribution $p(y_2 | y_1, X_1, X_2)$. Keep in mind that y_1 and y_2 are [jointly Gaussian](#) since they both come from the same multivariate distribution. Since they are jointly Gaussian and we have a finite number of samples we can write:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right)$$

Where:

$$\begin{aligned} \mu_1 &= m(X_1) \quad (n_1 \times 1) \\ \mu_2 &= m(X_2) \quad (n_2 \times 1) \\ \Sigma_{11} &= k(X_1, X_1) \quad (n_1 \times n_1) \\ \Sigma_{22} &= k(X_2, X_2) \quad (n_2 \times n_2) \\ \Sigma_{12} &= k(X_1, X_2) = k_{21}^\top \quad (n_1 \times n_2) \end{aligned}$$

Note that Σ_{11} is independent of Σ_{22} and vice versa.

We can then get the [conditional distribution](#):

$$\begin{aligned} p(y_2 | y_1, X_1, X_2) &= \mathcal{N}(\mu_{2|1}, \Sigma_{2|1}) \\ \mu_{2|1} &= \mu_2 + \Sigma_{21} \Sigma_{11}^{-1} (y_1 - \mu_1) \\ &= \Sigma_{21} \Sigma_{11}^{-1} y_1 \quad (\text{if assume mean prior } \mu = 0) \\ \Sigma_{2|1} &= \Sigma_{22} - \Sigma_{21} \Sigma_{11}^{-1} \Sigma_{12} \end{aligned}$$

We can write these as follows (Note here that $\Sigma_{11} = \Sigma_{11}^\top$ since it's [symmetric](#)):

$$\begin{aligned}\mu_{2|1} &= \Sigma_{21} \Sigma_{11}^{-1} \mathbf{y}_1 & \Sigma_{2|1} &= \Sigma_{22} - \Sigma_{21} \Sigma_{11}^{-1} \Sigma_{12} \\ &= (\Sigma_{11}^{-1} \Sigma_{12})^\top \mathbf{y}_1 & &= \Sigma_{22} - (\Sigma_{11}^{-1} \Sigma_{12})^\top \Sigma_{12}\end{aligned}$$

It is then possible to predict $\mathbf{y}_2$ corresponding to the input samples $\mathbf{X}_2$ by using the mean $\mu_{2|1}$ of the resulting distribution as a prediction. Notice that the mean of the posterior predictions $\mu_{2|1}$ of a Gaussian process are weighted averages of the observed variables $\mathbf{y}_1$, where the weighting is based on the covariance function k . The variance σ_2^2 of these predictions is then the diagonal of the covariance matrix $\Sigma_{2|1}$.

The Gaussian process posterior is implemented in the `GP` method below. We can compute the $\Sigma_{11}^{-1} \Sigma_{12}$ term with the help of Scipy's [solve](#) function ⁽²⁾.

```
# Gaussian process posterior
def GP(X1, y1, X2, kernel_func):
    """
    Calculate the posterior mean and covariance matrix for y2
    based on the corresponding input X2, the observations (y1,
    and the prior kernel function.
    """
    # Kernel of the observations
    Σ11 = kernel_func(X1, X1)
    # Kernel of observations vs to-predict
    Σ12 = kernel_func(X1, X2)
    # Solve
    solved = scipy.linalg.solve(Σ11, Σ12, assume_a='pos').T
    # Compute posterior mean
    μ2 = solved @ y1
    # Compute the posterior covariance
    Σ22 = kernel_func(X2, X2)
    Σ2 = Σ22 - (solved @ Σ12)
    return μ2, Σ2 # mean, covariance
```

The code below calculates the posterior distribution based on 8 observations from a sine function. The results are plotted below. The top figure shows the distribution where the red line is the posterior mean, the shaded area is the 95% prediction interval, the black dots are the observations (X_1, y_1) . The prediction interval is computed from the standard deviation $\sigma_{2|1}$, which is the square root of the diagonal of the covariance matrix. The bottom figure shows 5 realizations (sampled functions) from this distribution.

Note that the distribution is quite confident of the points predicted around the observations (X_1, y_1) , and that the prediction interval gets larger the further away it is from these points.

```
# Compute the posterior mean and covariance

# Define the true function that we want to regress on
f_sin = lambda x: (np.sin(x)).flatten()

n1 = 8 # Number of points to condition on (training points)
n2 = 75 # Number of points in posterior (test points)
ny = 5 # Number of functions that will be sampled from the po:
domain = (-6, 6)

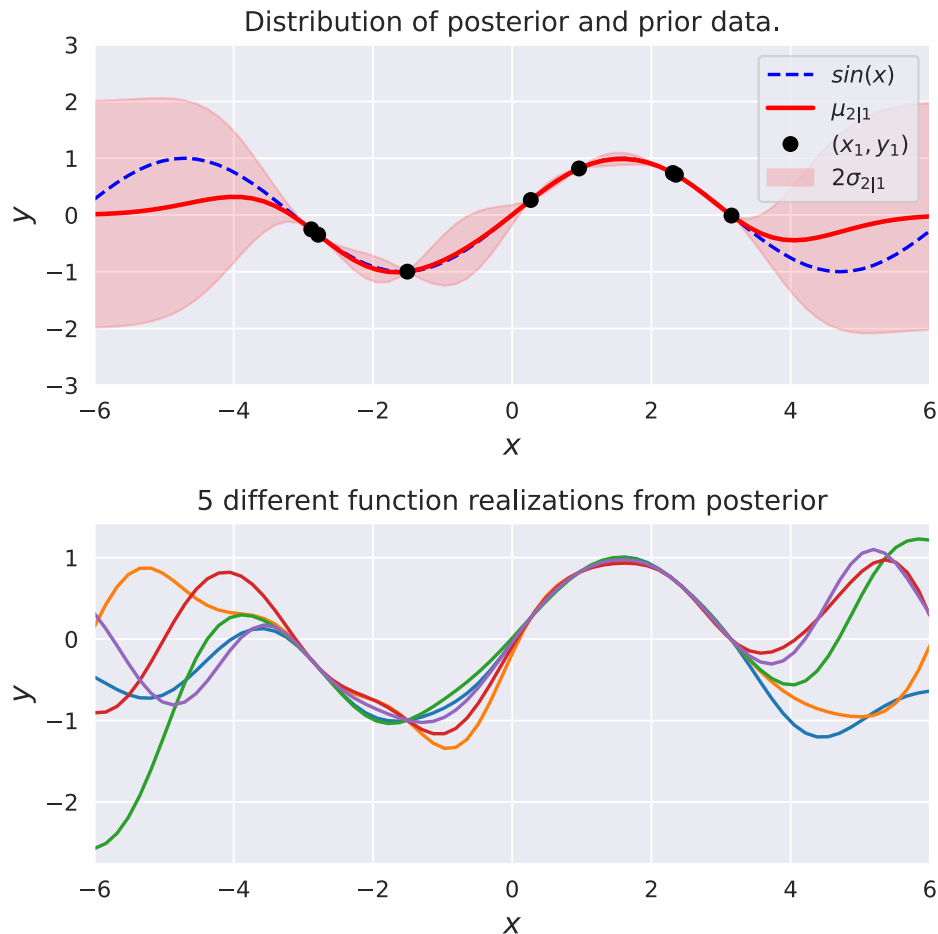
# Sample observations (X1, y1) on the function
X1 = np.random.uniform(domain[0]+2, domain[1]-2, size=(n1, 1))
y1 = f_sin(X1)
# Predict points at uniform spacing to capture function
X2 = np.linspace(domain[0], domain[1], n2).reshape(-1, 1)
# Compute posterior mean and covariance
mu2, Sigma2 = GP(X1, y1, X2, exponentiated_quadratic)
# Compute the standard deviation at the test points to be plot:
sigma2 = np.sqrt(np.diag(Sigma2))

# Draw some samples of the posterior
y2 = np.random.multivariate_normal(mean=mu2, cov=Sigma2, size=ny)
```

```

# Plot the posterior distribution and some samples
fig, (ax1, ax2) = plt.subplots(
    nrows=2, ncols=1, figsize=(6, 6))
# Plot the distribution of the function (mean, covariance)
ax1.plot(X2, f_sin(X2), 'b--', label='$sin(x)$')
ax1.fill_between(X2.flat,  $\mu_2 - 2\sigma_2$ ,  $\mu_2 + 2\sigma_2$ , color='red',
    alpha=0.15, label='$2 \sigma_{2|1}$')
ax1.plot(X2,  $\mu_2$ , 'r-', lw=2, label='$\mu_{2|1}$')
ax1.plot(X1, y1, 'ko', linewidth=2, label='$ (x_1, y_1) $')
ax1.set_xlabel('$x$', fontsize=13)
ax1.set_ylabel('$y$', fontsize=13)
ax1.set_title('Distribution of posterior and prior data.')
ax1.axis([domain[0], domain[1], -3, 3])
ax1.legend()
# Plot some samples from this function
ax2.plot(X2, y2.T, '-')
ax2.set_xlabel('$x$', fontsize=13)
ax2.set_ylabel('$y$', fontsize=13)
ax2.set_title('5 different function realizations from posterior')
ax1.axis([domain[0], domain[1], -3, 3])
ax2.set_xlim([-6, 6])
plt.tight_layout()
plt.show()
#

```



Noisy observations

The predictions made above assume that the observations $f(X_1) = \mathbf{y}_1$ come from a noiseless distribution. We can notice this in the plot above because the posterior variance becomes zero at the observations $(X_1, \mathbf{y}_1)$. We can make predictions from noisy observations $f(X_1) = \mathbf{y}_1 + \epsilon$, by modelling the noise ϵ as Gaussian noise with variance σ_ϵ^2 .

This noise can be modelled by adding it to the covariance kernel of our observations:

$$\Sigma_{11} = k(X_1, X_1) + \sigma_\epsilon^2 I$$

Where I is the identity matrix. Note that the noise only changes kernel values on the diagonal (white noise is independently distributed). The Gaussian process posterior with noisy observations is implemented in the `GP_noise` method below.

```

# Gaussian process posterior with noisy observations
def GP_noise(X1, y1, X2, kernel_func, σ_noise):
    """
    Calculate the posterior mean and covariance matrix for y2
    based on the corresponding input X2, the noisy observation:
    (y1, X1), and the prior kernel function.
    """

    # Kernel of the noisy observations
    Σ11 = kernel_func(X1, X1) + ((σ_noise ** 2) * np.eye(n1))
    # Kernel of observations vs to-predict
    Σ12 = kernel_func(X1, X2)
    # Solve
    solved = scipy.linalg.solve(Σ11, Σ12, assume_a='pos').T
    # Compute posterior mean
    μ2 = solved @ y1
    # Compute the posterior covariance
    Σ22 = kernel_func(X2, X2)
    Σ2 = Σ22 - (solved @ Σ12)
    return μ2, Σ2 # mean, covariance

```

The code below calculates the posterior distribution of the previous 8 samples with added noise. Note in the plots that the variance $\sigma_{2|1}^2$ at the observations is no longer 0, and that the functions sampled don't necessarily have to go through these observational points anymore.

```

# Compute the posterior mean and covariance

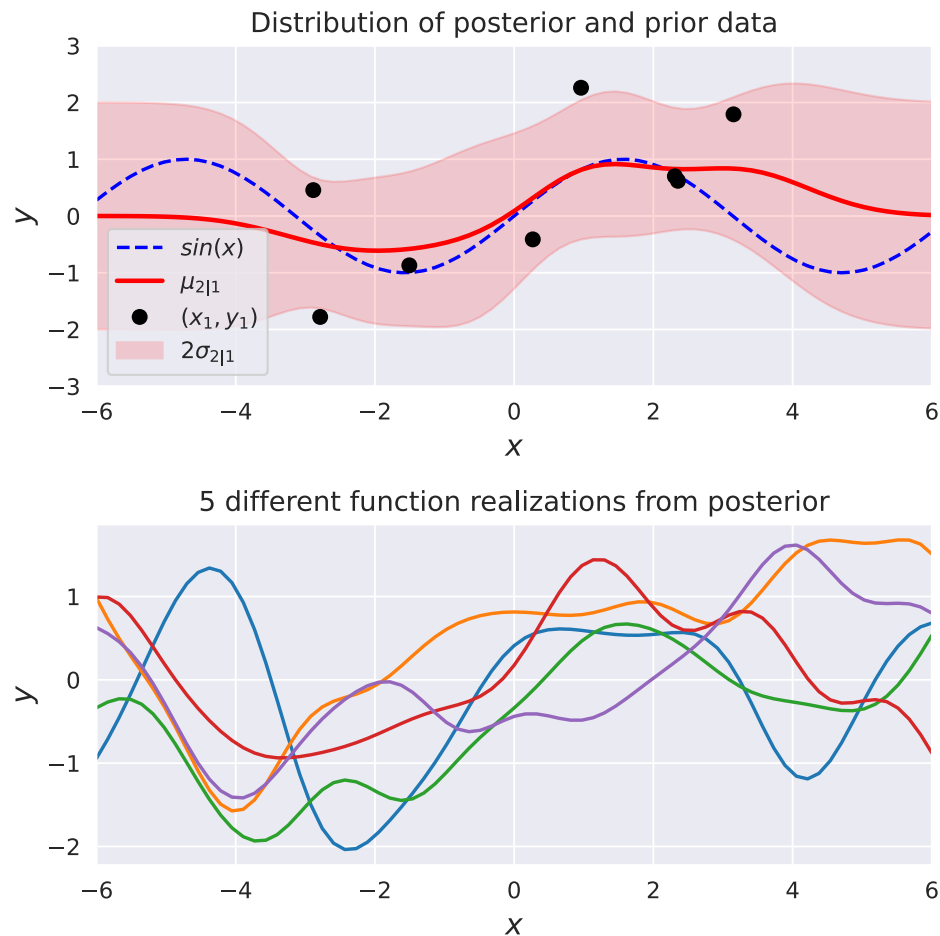
σ_noise = 1. # The standard deviation of the noise
# Add noise kernel to the samples we sampled previously
y1 = y1 + ((σ_noise ** 2) * np.random.randn(n1))

# Compute posterior mean and covariance
μ2, Σ2 = GP_noise(X1, y1, X2, exponentiated_quadratic, σ_noise)
# Compute the standard deviation at the test points to be plot
σ2 = np.sqrt(np.diag(Σ2))

# Draw some samples of the posterior
y2 = np.random.multivariate_normal(mean=μ2, cov=Σ2, size=ny)

```

```
# Plot the posterior distribution and some samples
```



To conclude we've implemented a Gaussian process and illustrated how to make predictions using its posterior distribution.

Key points to take away are:

- A Gaussian process is a distribution over functions fully specified by a mean and covariance function.
- Every finite set of the Gaussian process distribution is a multivariate Gaussian.
- The posterior predictions of a Gaussian process are weighted averages of the observed data where the weighting is based on the covariance and mean functions.

This was the first post part of a series on Gaussian processes. You can read [how to fit a Gaussian process kernel in the follow-up post](#).

1. [Understanding Gaussian processes \(this\)](#).
2. [Fitting a Gaussian process kernel](#)
3. [Gaussian process kernels](#)

Sidenotes

1. Observe in the plot of the 41D Gaussian marginal from the exponentiated quadratic prior that the functions drawn from the Gaussian process distribution can be non-linear. The non-linearity is because the kernel can be interpreted as implicitly computing the inner product in a different space than the original input space (e.g. a higher dimensional feature space). This is what is commonly known as the [kernel trick](#).
2. $\Sigma_{11}^{-1} \Sigma_{12}$ can be computed with the help of Scipy's [solve](#) function, which solves for x the [linear system](#) $\Sigma_{11} \cdot x = \Sigma_{12}$. Using this method [improves](#) the speed and numerical accuracy compared to computing the inverse of Σ_{11} directly. Especially since it can make use of the fact that Σ_{11} is [symmetric and positive definite](#).

References

1. [Introduction to Gaussian processes video lecture](#) by Nando de Freitas.
2. [Gaussian Processes for Machine Learning](#) by Carl Edward Rasmussen and Christopher K. I. Williams (Book covering Gaussian processes in detail, online version downloadable as pdf).
3. [Stochastic Processes and Applications](#) by Grigorios A. Pavliotis.

```
# Python package versions used
```



```
Python implementation: CPython
```

```
Python version       : 3.9.4
```

```
IPython version      : 7.23.1
```

```
matplotlib: 3.4.2
```

```
numpy       : 1.20.2
```

```
scipy       : 1.6.3
```

```
seaborn     : 0.11.1
```

This post at peterroelants.github.io is generated from an Python notebook file. [Link to the full IPython notebook file](#)

Originally published on January 5, 2019.

 Gaussian Process Gaussian Distribution Probability Kernel NumPy Machine Learning Notebook